

DOCUMENTAZIONE TECNICA

API Reference

Contratto REST e WebSocket, codici di errore e matrice dei permessi



Campo	Valore
Applicativo	Guest Portal — Cisco Catalyst 9800 Guest Management Desk
Indirizzo (produzione)	https://guestportal.dompe.com
Documento	API — API Reference
Versione	1.2
Data di emissione	25 settembre 2026
Stato	Emesso
Destinatari	IT Infrastructure, Security, Sviluppo, Service Desk
Classificazione	Uso interno
Repository di riferimento	github.com/ACF-Dompe/Centralino
Revisione del codice	main · 3338398 (+ modifiche non ancora committate)

Indice

1	Convenzioni.....	3
1.1	Trasporto e formato	3
1.2	Forma delle risposte	3
1.3	Codici di errore	3
2	Sonde di salute.....	5
3	Autenticazione	6
4	Sessione	8
5	Sedi.....	9
6	Controller	10
7	Directory aziendale.....	11
8	Ospiti	12
9	Configurazioni e storico.....	14
10	Amministrazione.....	15
10.1	Directory utenti.....	15
10.2	Sedi.....	15
10.2.1	Password del controller.....	16
10.3	Account di emergenza.....	17
11	WebSocket	18
12	Matrice permessi	19
13	Note di compatibilità	21

1 Convenzioni

Tutta la superficie applicativa è esposta sotto `/api`, sulla stessa origine del frontend. L'instradamento fra le due Container App è compito dell'Application Gateway: `/api/*` al backend, tutto il resto al frontend.

1.1 Trasporto e formato

Aspetto	Valore
Base	Produzione: <code>https://guestportal.dompe.com/api</code> Collaudo: <code>https://guestportal-stg.dompe.com/api</code> Sviluppo: <code>https://guestportal-dev.dompe.com/api</code>
Formato	JSON in richiesta e risposta; <code>application/x-www-form-urlencoded</code> per i soli endpoint SAML
Limite del corpo	1 MB
CORS	Assente per scelta: le richieste sono sempre di pari origine
Autenticazione	Cookie di sessione <code>guestportal.sid</code> (HttpOnly, SameSite=lax, Secure in produzione)
Correlazione	Ogni risposta porta <code>X-Request-Id</code> : è la chiave per ritrovare la richiesta nei log
Compressione	Attiva

Tabella 1 — Convenzioni di trasporto.

1.2 Forma delle risposte

```
// successo – lettura
{ "data": <payload> }

// successo – comando
{ "success": true, "data": <payload opzionale> }

// errore
{ "success": false, "error": "<codice stabile>", "message": "<testo per l'operatore>" }
```

Il **codice** è il contratto: stabile, pensato per essere confrontato dal client. Il **messaggio** è per l'operatore e può cambiare. Alcune risposte di errore portano campi aggiuntivi (`requiredRoles`, `count`, `credentialSecretName`) utili a costruire un messaggio più preciso.

1.3 Codici di errore

Codice	HTTP	Significato	Dove
<code>unauthenticated</code>	401	Nessuna sessione valida.	Ovunque
<code>user_not_provisioned</code>	403	Autenticato ma non ancora profilato.	Ovunque
<code>user_suspended</code>	403	Profilo sospeso.	Ovunque
<code>insufficient_role</code>	403	Ruolo insufficiente; la risposta elenca <code>requiredRoles</code> .	Ovunque
<code>authz_unavailable</code>	503	Risoluzione dei permessi fallita.	Ovunque
<code>sede_required</code>	400	L'operazione richiede una sede.	WLC, ospiti

Codice	HTTP	Significato	Dove
<code>sede_forbidden</code>	403	Sede non concessa.	WLC, ospiti, sedi
<code>sede_mismatch</code>	403	La sede richiesta non è quella della sessione.	GET /guests
<code>NO_SEDE_SELECTED</code>	409	Nessuna sede in sessione.	Ospiti, config
<code>SEDE_INACTIVE</code>	409	Sede fuori servizio.	WLC, sessione
<code>WLC_NOT_CONFIGURED</code>	400	Sede priva di indirizzo del controller.	WLC, test sede
<code>CREDENTIAL_MISSING</code>	400	Manca <code>WLC_PASSWORD_<CODICE></code> .	WLC, test sede
<code>invalid_payload</code>	400	Corpo non valido.	Amministrazione
<code>invalid_code</code>	400	Codice sede fuori formato.	Creazione sede
<code>duplicate_code</code>	409	Codice sede già in uso.	Creazione sede
<code>credential_collision</code>	409	Due codici normalizzano sulla stessa variabile d'ambiente.	Creazione sede
<code>immutable_code</code>	409	Il codice sede non è modificabile.	Modifica sede
<code>untested_sede</code>	409	Attivazione senza un test riuscito; si può forzare.	Attivazione sede
<code>SEDE_HAS_GUESTS</code>	409	Eliminazione rifiutata: la sede ha ospiti.	Eliminazione sede
<code>auto_admin_immutable</code>	409	Ruolo e stato di un amministratore per convenzione non sono modificabili.	Profilazione utente
<code>cannot_modify_self</code>	409	Non si può cambiare il proprio ruolo o stato.	Profilazione utente
<code>last_admin</code>	409	È l'ultimo amministratore attivo.	Profilazione, eliminazione utente
<code>last_breakglass</code>	409	È l'ultimo account di emergenza utilizzabile.	Disabilitazione emergenza
<code>unknown_sede</code>	400	Sede inesistente fra quelle concesse.	Profilazione utente
<code>invalid_query</code>	400	Ricerca fuori dai limiti (2–64 caratteri) o con caratteri di controllo.	Directory
<code>directory_unavailable</code>	503	Ricerca nella directory non abilitata.	Directory
<code>directory_error</code>	502	Microsoft Graph non ha risposto o ha rifiutato.	Directory
<code>keyvault_unavailable</code>	503	<code>KEY_VAULT_URL</code> non impostato.	Password sede, ricarica
<code>keyvault_error</code>	502	Chiamata a Key Vault fallita (per una scrittura, tipicamente il ruolo mancante).	Password sede, ricarica
<code>secret_not_found</code>	404	Il segreto della sede non esiste nel vault.	Password sede
<code>not_found</code>	404	Risorsa inesistente.	Amministrazione

Tabella 2 — Codici di errore applicativi.

Il WebSocket vive sullo stesso host: in produzione <wss://guestportal1.dompe.com/api/ws>.

2 Sonde di salute

Aperte, senza autenticazione: la piattaforma deve poter valutare lo stato del container.

Metodo	Percorso	Ruolo	Descrizione
GET	/api/healthz	—	Liveness. Risponde sempre 200 se il processo è vivo; non verifica dipendenze.
GET	/api/readyz	—	Readiness. Verifica la connettività a PostgreSQL: 200 se pronto, 503 se il database non risponde.
GET	/api/health	—	Deprecato. Equivalente a /api/healthz , mantenuto per compatibilità.

```
GET /api/readyz → 200
{ "status": "ok", "db": "connected", "uptime": 3421.7, "timestamp": "2026-09-18T07:12:00.000Z" }
```

3 Autenticazione

Metodo	Percorso	Ruolo	Descrizione
GET	/api/auth/login	—	Avvia l'accesso SSO: reindirizza a Entra ID. Il parametro <code>redirect</code> è accettato solo se punta alla stessa origine. Risponde 501 se SSO non è utilizzabile.
POST	/api/auth/callback	—	Assertion Consumer Service. Riceve la risposta SAML in <code>application/x-www-form-urlencoded</code> , crea la sessione, esegue il provisioning just-in-time e reindirizza.
POST	/api/auth/logout	autenticato	Distrukge la sessione locale; con SLO configurato reindirizza all'endpoint di disconnessione dell'IdP. Una sessione di emergenza esce solo localmente.
POST	/api/auth/slo/callback	—	Riceve la conferma di disconnessione dall'IdP e riporta alla radice.
GET	/api/auth/me	—	Profilo corrente. 200 anche per un utente non abilitato : è così che l'interfaccia sa perché è bloccato. 401 se non autenticato, 404 se SSO non è configurato.
GET	/api/auth/breakglass/status	—	Indica se l'accesso di emergenza è utilizzabile da questo client . Risponde <code>false</code> sia se è spento sia se l'indirizzo è fuori lista.
POST	/api/auth/breakglass/login	—	Accesso locale di emergenza. 404 se spento o fuori lista, 429 se limitato, 401 con messaggio generico in ogni caso di rifiuto.

Perché /api/auth/me risponde 200 a un utente bloccato

È l'unico endpoint che deve poter spiegare a un utente autenticato perché non può fare nulla. Rispondere 403 lascerebbe all'interfaccia solo la schermata di accesso, che l'utente completerebbe con successo, all'infinito.

RISPOSTA DI /API/AUTH/ME

```
{
  "success": true,
  "data": {
    "nameID": "...", "upn": "m.rossi@dompe.com", "email": "m.rossi@dompe.com",
    "displayName": "Mario Rossi", "givenName": "Mario", "surname": "Rossi",
    "objectId": "9f3c...", "authMethod": "saml",
    "role": "operator",           // risolto per richiesta, mai letto dalla sessione
    "status": "active",         // pending | active | suspended
    "sedeIds": [1, 2]           // sempre una lista concreta, mai un jolly
  }
}
```

● Accesso SSO SAML 2.0 e provisioning just-in-time

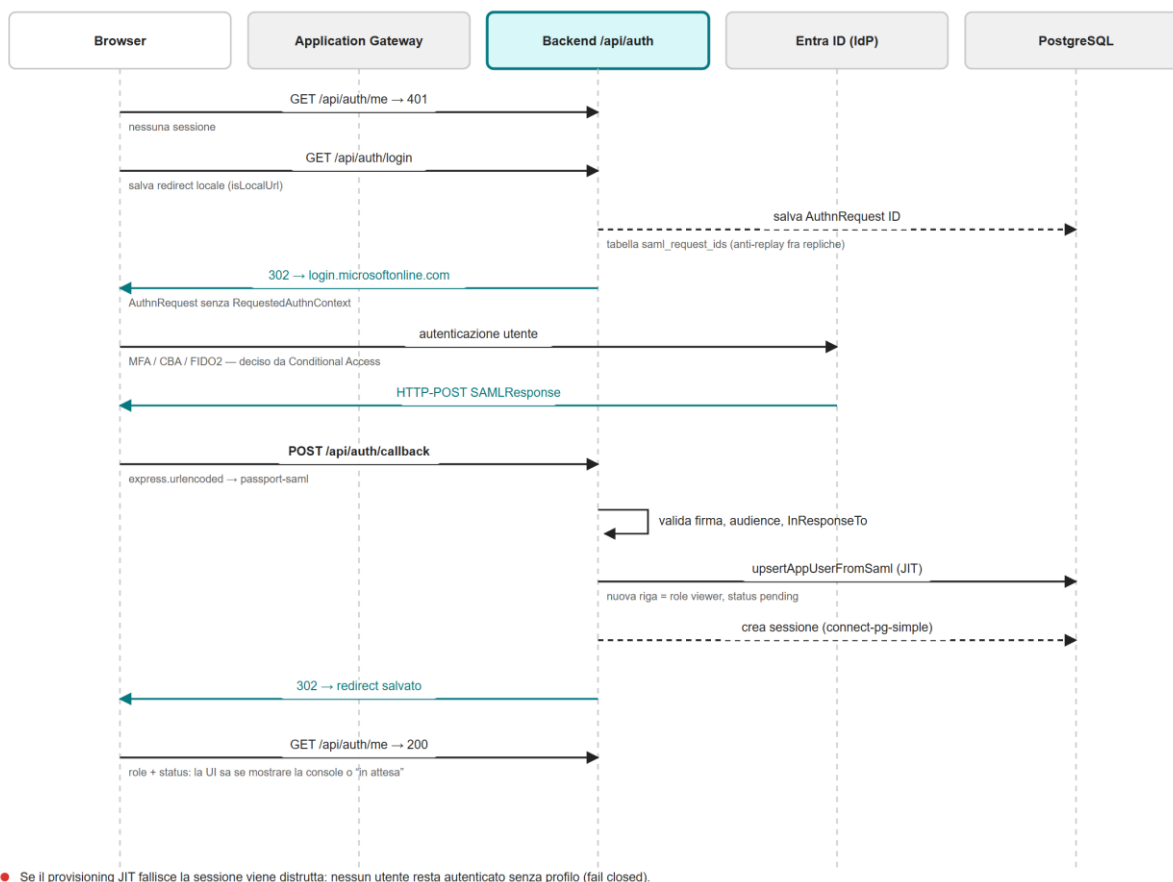


Figura 1 — Sequenza dell'accesso SSO e del provisioning just-in-time.

4 Sessione

Metodo	Percorso	Ruolo	Descrizione
GET	/api/session/context	autenticato attivo	Bootstrap del client: chi è connesso, che cosa può fare, su quale sede si trova. Può includere <code>notice</code> : "SEDE_INACTIVE".
DELETE	/api/session/sede	autenticato attivo	«Cambia sede»: libera la sede mantenendo la sessione. Risponde 204.

```
GET /api/session/context → 200
{
  "data": {
    "user": { "displayName", "email", "role", "status", "sedeIds": [1,2] | null },
    "sede": { "id", "code", "name", "city", "address", "active", "wlcSsid", ... },
    "wlc": { "host", "port", "sshPort", "username", "wlanSsid", "sedeId",
              "usable": true,           // il server può parlare con il controller
              "authenticated": true    // QUESTA sessione è legata alla sede
            }
  }
}
```

Per un utente non amministratore i campi `host`, `port`, `sshPort` e `username` della sede sono **rimossi** dalla risposta: un operatore non ne ha bisogno, e toglierli dalla vista è il motivo per cui sono stati spostati nel pannello di amministrazione. Lo SSID resta, perché è il nome di rete che l'ospite deve digitare. `sedeIds` vale `null` per un amministratore, a indicare «nessuna restrizione».

5 Sedi

Metodo	Percorso	Ruolo	Descrizione
GET	/api/sedi	autenticato attivo	Sedi in servizio che il chiamante può raggiungere. Per i non amministratori i parametri del controller sono rimossi.
GET	/api/sedi/:id	autenticato attivo	Una sede, se il chiamante vi è abilitato.

6 Controller

Metodo	Percorso	Ruolo	Descrizione
POST	/api/wlc/login	admin · operator	Lega la sessione a una sede. Accetta solo { <code>sedeId</code> }: indirizzo, porta e account vengono dal record di sede, la password da Key Vault.
POST	/api/wlc/create-user	admin	Creazione diretta di un utente sul controller.
PUT	/api/wlc/status-user	admin	Abilitazione o disabilitazione di un utente sul controller.
POST	/api/wlc/delete-user	admin	Rimozione di un utente dal controller.
POST	/api/wlc/get-users	admin	Elenco degli utenti presenti sul controller.
POST	/api/wlc/import-users	admin	Importa nella banca dati locale gli utenti trovati sul controller e non ancora presenti.

```
POST /api/wlc/login
{ "sedeId": 1 }
```

```
→ 200 { "success": true, "status": 200, "authMethod": "webui" }
→ 400 { "success": false, "error": "CREDENTIAL_MISSING", "message": "Password WLC non configurata ..." }
→ 400 { "success": false, "error": "WLC_NOT_CONFIGURED", ... }
→ 409 { "success": false, "error": "SEDE_INACTIVE", ... }
→ 404 { "success": false, "error": "Sede non trovata" }
```

Solo l'identificativo della sede attraversa la rete

In precedenza questo endpoint accettava indirizzo, porta, account e password dal corpo della richiesta e li riscriveva in banca dati: la schermata di accesso era, di fatto, un modo per riconfigurare qualunque controller. Oggi il corpo contiene solo `sedeId`.

Gli endpoint `create-user`, `status-user`, `delete-user` e `get-users` accettano ancora i parametri di connessione nel corpo. Sono riservati al ruolo `admin` e i valori passano tutti dalla validazione che impedisce l'iniezione di comandi IOS-XE (nessun capoverso, nessun byte nullo, alfabeto ristretto).

7 Directory aziendale

Metodo	Percorso	Ruolo	Descrizione
GET	/api/directory/users?q=	admin · operator	Ricerca di persone in Entra ID per il campo Referente . Interroga Microsoft Graph a ogni chiamata: nessuna cache , né sul server né nel browser.

```
GET /api/directory/users?q=bian → 200
{ "success": true, "data": [
  { "id": "9f3c...", "displayName": "Anna Bianchi" },
  { "id": "2a71...", "displayName": "Marco Bianchini" }
]}
```

```
→ 400 invalid_query      testo < 2 o > 64 caratteri, o caratteri di controllo
→ 503 directory_unavailable ricerca non abilitata (DIRECTORY_SEARCH_ENABLED)
→ 502 directory_error     Graph non ha risposto o ha rifiutato
```

- Escono **solo** **id** e **displayName**: nessun indirizzo, nessun UPN, nessun altro attributo della directory.
- Sono restituiti solo utenti **abilitati** il cui UPN appartiene ai domini di **DIRECTORY_UPN_DOMAINS**, verificati sia nel filtro sia sui risultati.
- Risposta con **Cache-Control: no-store**. **Il testo cercato non viene mai registrato**: è il nome di una persona.
- Al massimo **DIRECTORY_MAX_RESULTS** risultati (15 per impostazione predefinita), ordinati per nome.

Un 503 non blocca la registrazione

Quando la ricerca è spenta — la condizione predefinita, e quella dello sviluppo locale — il campo **Referente** si comporta come un normale campo di testo. La registrazione di un ospite non dipende in alcun modo da questo endpoint.

8 Ospiti

Metodo	Percorso	Ruolo	Descrizione
GET	/api/guests	autenticato attivo	Ospiti della sede in sessione . Filtri <code>?search</code> e <code>?status</code> . Ogni ospite porta <code>connected: true false null</code> . Un <code>?sedeId</code> diverso da quello di sessione produce 403 <code>sede_mismatch</code> .
POST	/api/guests	admin · operator	Registra un ospite. Restituisce <code>oneTimePassword</code> : l'unica volta in cui la password è leggibile.
PUT	/api/guests/:id	admin · operator	Modifica un ospite. Un cambio di stato si riflette sul controller della sua sede.
DELETE	/api/guests/:id	admin · operator	Elimina l'ospite e invia <code>no user-name</code> al controller.
POST	/api/guests/:id/resend-credentials	admin · operator	Rigenera la password, la spinge sul controller e la reinvia. Lo username resta invariato.

CREAZIONE

```
POST /api/guests
{
  "name": "Mario Rossi",
  "host": "Anna Bianchi",           // referente interno – obbligatorio
  "durationMinutes": 240,           // 1 ... 10080 (una settimana)
  "sedeId": 1,                      // obbligatorio, e deve essere fra quelle concesse
  "email": "mario.rossi@example.com", // facoltativo: se presente, invio automatico
  "phone": null, "company": "ACME", "remarks": "Riunione commerciale"
}

→ 200
{ "data": { ...ospite..., "oneTimePassword": "Kt7mRq4Wnx2F" } }
```

`oneTimePassword` compare una volta sola

Non viene scritta in banca dati e non esiste un endpoint per rileggerla. Il percorso di recupero è `POST /api/guests/:id/resend-credentials`, che ne genera una nuova.

REINVIO DELLE CREDENZIALI

```
POST /api/guests/:id/resend-credentials → 200
{
  "success": true,
  "oneTimePassword": "Kt7mRq4Wnx2F",
  "wlcUpdated": true,           // il controller ha accettato i comandi
  "wlcIssues": [ "lifetime registrato 240 min invece di 480" ],
                                     // presente solo se la riletture ha trovato scostamenti
  "emailSent": true,
  "emailMode": "graph"         // "graph" | "demo-log"
}

→ 400 se l'ospite non ha un indirizzo e-mail
```

Il reinvio ricrea l'account e fa ripartire la durata

Il controller conta la durata da `creation-time`, che una modifica **non** azzerà: le credenziali reinviolate per un account vecchio arrivavano già scadute, mentre il messaggio prometteva una durata nuova. Ora l'account viene eliminato e ricreato, e il conteggio nell'applicazione riparte di conseguenza.

• Registrazione di un ospite e consegna delle credenziali

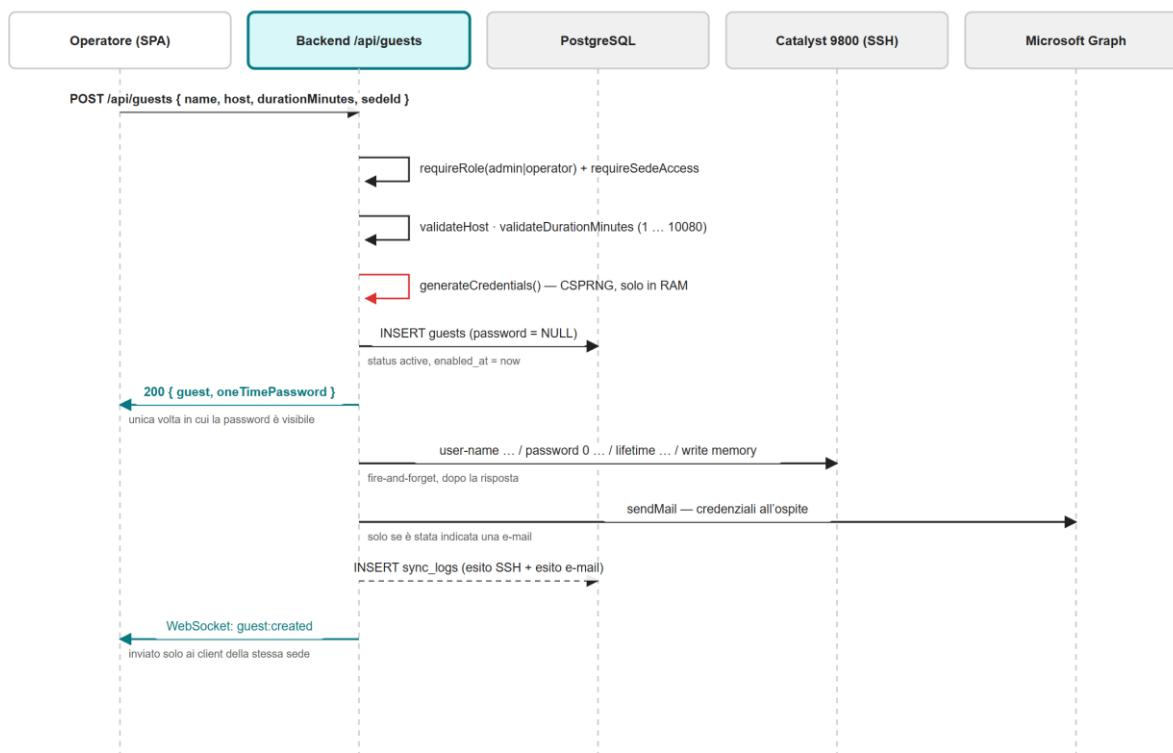


Figura 2 — Sequenza della registrazione di un ospite.

9 Configurazioni e storico

Metodo	Percorso	Ruolo	Descrizione
GET	/api/config/wlc	autenticato attivo	Deprecato. Risolve la sede dalla sessione. Usare /api/session/context . La password è sempre omessa.
GET	/api/config/sms	admin	Configurazione del canale SMS. La chiave del provider è omessa dalla risposta.
PUT	/api/config/sms	admin	Aggiorna la configurazione SMS. Funzione attualmente dormiente.
GET	/api/sync-logs	admin · operator	Ultime 200 voci di storico.
DELETE	/api/sync-logs	admin	Svuota lo storico.

10 Amministrazione

Tutto ciò che segue è montato dietro `ensureAuthenticated + loadAuthorization + requireRole('admin')`. Nessuna rotta al suo interno ricontrolla il ruolo: il controllo è unico e sta all'ingresso.

10.1 Directory utenti

Metodo	Percorso	Ruolo	Descrizione
GET	/api/admin/users	admin	Elenco della directory. Filtri <code>?status</code> e <code>?search</code> .
GET	/api/admin/users/:id	admin	Una voce della directory.
PATCH	/api/admin/users/:id	admin	Profila un utente: { <code>role</code> , <code>status</code> , <code>sedeIds</code> }.
DELETE	/api/admin/users/:id	admin	Rimuove la voce. Verrà ricreata, bloccata, al prossimo accesso dell'utente.
Rifiuto	HTTP	Quando	
<code>auto_admin_immutable</code>	409	Si tenta di cambiare ruolo o stato — o di eliminare — un amministratore per convenzione. Eliminarlo non otterrebbe nulla: al prossimo accesso tornerebbe amministratore attivo. Le sedi si possono comunque concedere: semplicemente non cambiano nulla.	
<code>cannot_modify_self</code>	409	Si tenta di cambiare il proprio ruolo o stato. Cambiare le proprie sedi è consentito.	
<code>last_admin</code>	409	La modifica lascerebbe la directory senza amministratori attivi.	
<code>unknown_sede</code>	400	<code>sedeIds</code> contiene una sede inesistente.	

Tabella 3 — Rifiuti sulla profilazione degli utenti.

Ogni modifica di profilo è registrata a livello `warn` con lo stato prima e dopo, così un alert può sorvegliare le promozioni ad amministratore senza bisogno di una regola dedicata.

10.2 Sedi

Metodo	Percorso	Ruolo	Descrizione
GET	/api/admin/sedi	admin	Tutte le sedi, con diagnostica e nomi del segreto e della variabile d'ambiente.
GET	/api/admin/sedi/:id	admin	Una sede con la stessa vista estesa.
POST	/api/admin/sedi	admin	Crea una sede. Nasce fuori servizio .
PUT	/api/admin/sedi/:id	admin	Aggiorna la sede. Il codice non è modificabile .
PATCH	/api/admin/sedi/:id/active	admin	Mette in servizio o fuori servizio: { <code>active</code> , <code>force?</code> }.
POST	/api/admin/sedi/:id/test	admin	Prova la connessione al controller. Solo diagnostica .
DELETE	/api/admin/sedi/:id	admin	Elimina la sede. Rifiutata finché ha ospiti.
GET	/api/admin/sedi/:id/password	admin	Legge dal vault la password del controller. Deviazione D4 : tracciata, una sede alla volta, <code>no-store</code> .
PUT	/api/admin/sedi/:id/password	admin	Sostituisce la password creando una nuova versione del segreto. Non restituisce mai il valore.

Metodo	Percorso	Ruolo	Descrizione
POST	/api/admin/wlc/reload	admin	Rilegge da Key Vault la password di ogni sede, senza riavvio.

VINCOLI SUL CODICE DI SEDE

- Formato `^[A-Z0-9]{2,20}$`.
- Univoco fra le sedi esistenti.
- Due codici non possono normalizzare sulla **stessa** variabile d'ambiente: condividerebbero la password (`credential_collision`).
- Non modificabile dopo la creazione: è ciò che lega la sede al proprio segreto in Key Vault.

ATTIVAZIONE

```

PATCH /api/admin/sedi/:id/active
{ "active": true }

→ 409 { "error": "untested_sede",
      "message": "Esegui un test di connessione riuscito prima di attivare la sede, ...",
      "credentialConfigured": false,
      "credentialSecretName": "WLC-PASSWORD-BCN" }

// per procedere comunque:
{ "active": true, "force": true }

```

Un test non deve avere effetti collaterali

La prova di connessione registra soltanto l'esito sulla sede. La versione precedente chiamava l'endpoint di accesso e poi scriveva `authenticated: true`: eseguire un test cambiava quale controller l'applicazione considerava attivo. Un test con un effetto collaterale non è un test.

10.2.1 Password del controller

Questi tre endpoint sono la **deviazione D4**: ribaltano, su richiesta esplicita, la regola per cui nessun segreto attraversa la API. Sono descritti per intero nel documento SEC.

```

GET /api/admin/sedi/1/password → 200 Cache-Control: no-store
{ "success": true, "data": { "password": "..." } }
→ 404 secret_not_found      il segreto non esiste nel vault

PUT /api/admin/sedi/1/password
{ "password": "nuova-password" } → 200 { "success": true }
→ 400 invalid_payload      spazi iniziali/finali, o caratteri non ammessi

POST /api/admin/wlc/reload → 200
{ "success": true, "data": {
  "loaded": ["MIL", "AQ"],
  "missing": ["BCN"],
  "failed": [{ "code": "NA", "error": "403 Forbidden: ..." }] } }

```

- La password è validata come ogni credenziale destinata a una sessione IOS-XE: ASCII stampabile, nessun a capo. Sono rifiutati anche gli spazi iniziali o finali, che il validatore rimuoverebbe in silenzio producendo un valore diverso da quello impostato sul controller.
- **L'applicazione non tocca il controller**: la password va cambiata prima sull'apparato, poi salvata qui.
- Una sede che fallisce durante la ricarica **conserva** il valore precedente: un errore transitorio non mette fuori servizio un controller funzionante.
- Ogni lettura, scrittura e ricarica produce una riga di audit a livello `warn` e una voce nello storico.

10.3 Account di emergenza

Metodo	Percorso	Ruolo	Descrizione
GET	/api/admin/breakglass	admin	Elenco degli account con stato, blocco e scadenza. Anche la sola lettura è registrata.
POST	/api/admin/breakglass/:u/enable	admin	Riabilita un account.
POST	/api/admin/breakglass/:u/disable	admin	Disabilita un account. Rifiutato se è l'ultimo utilizzabile (<i>last_breakglass</i>).
POST	/api/admin/breakglass/:u/unlock	admin	Azzera un blocco da tentativi falliti.

Non esiste un endpoint per creare un account di emergenza

Né per ruotarne la password. Quelle operazioni vivono solo in `make breakglass`, perché una sessione amministrativa compromessa non deve poter coniare una credenziale che aggira Entra ID e l'accesso condizionale.

11 WebSocket

Aspetto	Valore
Endpoint	<code>wss://<host>/api/ws</code> — in produzione <code>wss://guestportal.dompe.com/api/ws</code>
Autenticazione	Cookie di sessione, verificato sull'upgrade; firma controllata e sessione letta dallo stesso store della API.
Autorizzazione	Risolta sull'upgrade: un utente sospeso non ottiene la connessione.
Ambito	Ogni connessione riceve solo gli eventi della propria sede. Amministratori e sessioni di emergenza ricevono tutto.
Rifiuto	HTTP/1.1 401 Unauthorized e chiusura del socket.
Formato	<code>{ type, data, timestamp }</code>

Tabella 4 — Caratteristiche del canale WebSocket.

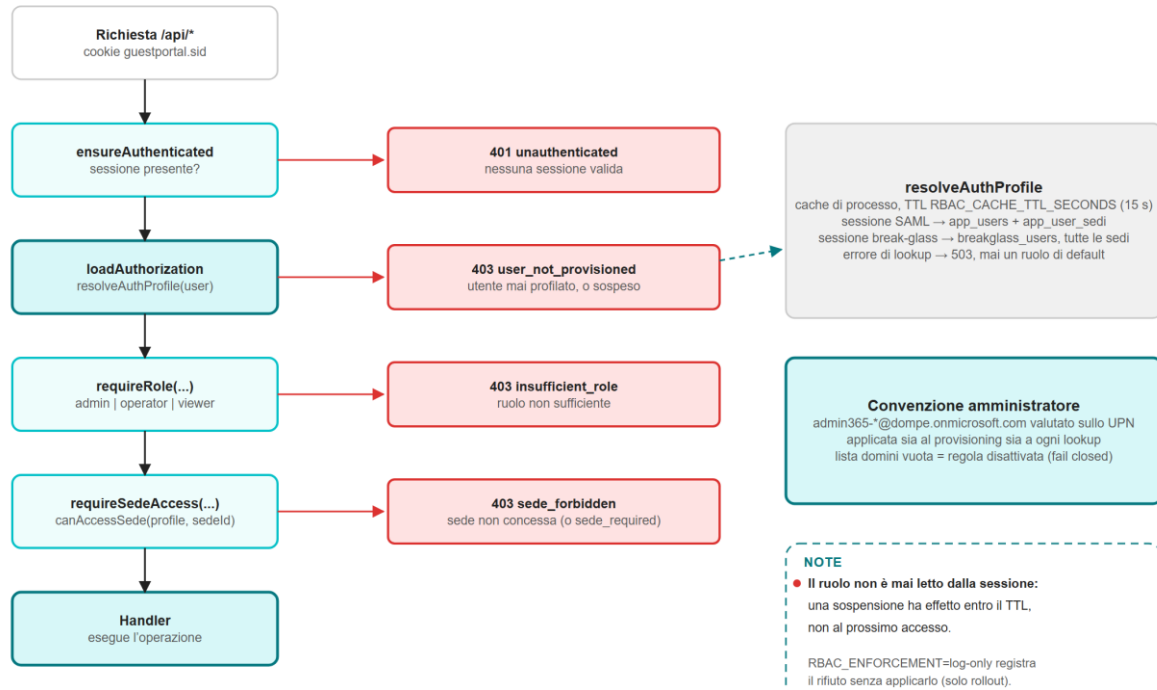
```

{ "type": "hello",          "data": { "server": "centralino-ws" },      "timestamp": "..."
}
{ "type": "guest:created",  "data": { id, name, username, sedeId },    "timestamp": "..."
}
{ "type": "guest:updated",  "data": { id, name, username, status, sedeId }, "timestamp": "..."
}
{ "type": "guest:expired",  "data": { id, name, username, sedeId },    "timestamp": "..."
}
{ "type": "guest:deactivated", "data": { id, name, username, sedeId },    "timestamp": "..."
}
{ "type": "guest:deleted",  "data": { id, name, username, sedeId },    "timestamp": "..."
}
{ "type": "guest:imported", "data": { id, name, username, sedeId },    "timestamp": "..."
}
{ "type": "sync:completed", "data": { sedeId },                        "timestamp": "..."
}

```

12 Matrice permessi

• Catena di autorizzazione di una richiesta



• Anche l'upgrade WebSocket passa dallo stesso controllo: un utente sospeso perde il socket, non lo conserva fino alla scadenza del cookie.

Figura 3 — Catena di controlli attraversata da ogni richiesta.

Endpoint	viewer	operator	admin
GET /api/healthz · /readyz	aperto	aperto	aperto
GET /api/auth/me	sì	sì	sì
POST /api/auth/breakglass/login	aperto (se abilitato)	aperto (se abilitato)	aperto (se abilitato)
GET /api/session/context	sì	sì	sì
DELETE /api/session/sede	sì	sì	sì
GET /api/sedi · /sedi/:id	solo concesse	solo concesse	tutte
POST /api/wlc/login	no	solo concesse	tutte
POST /api/wlc/* (gestione utenti)	no	no	sì
GET /api/guests	sede in sessione	sede in sessione	sede in sessione
POST · PUT · DELETE /api/guests	no	solo concesse	tutte
POST /api/guests/:id/resent-credentials	no	solo concesse	tutte
GET /api/directory/users	no	sì	sì
GET /api/sync-logs	no	sì	sì
DELETE /api/sync-logs	no	no	sì
GET · PUT /api/config/sms	no	no	sì
/api/admin/**	no	no	sì

Tabella 5 — Matrice endpoint × ruolo. «solo concesse» indica le sole sedi assegnate all'utente.

Una sessione di **emergenza** ha il ruolo registrato sul proprio account (per default [admin](#)) e raggiunge tutte le sedi: non dipende da [app_users](#), perché è l'accesso che deve funzionare quando il resto è rotto.

13 Note di compatibilità

Elemento	Stato	Sostituto
GET /api/health	Deprecato	GET /api/healthz
GET /api/config/wlc	Deprecato	GET /api/session/context
PUT /api/config/wlc	Rimosso	Pannello di amministrazione (PUT /api/admin/sedi/:id)
GET · PUT /api/config/email	Rimosso	Nessuno: la posta passa solo da Microsoft Graph
Parametri del controller in POST /api/wlc/login	Rimossi	Record di sede + Key Vault
?sedeId su GET /api/guests	Tollerato	La sede della sessione. Un valore diverso produce 403.

Tabella 6 — Elementi deprecati e rimossi.